Graphs & Graph Traversal

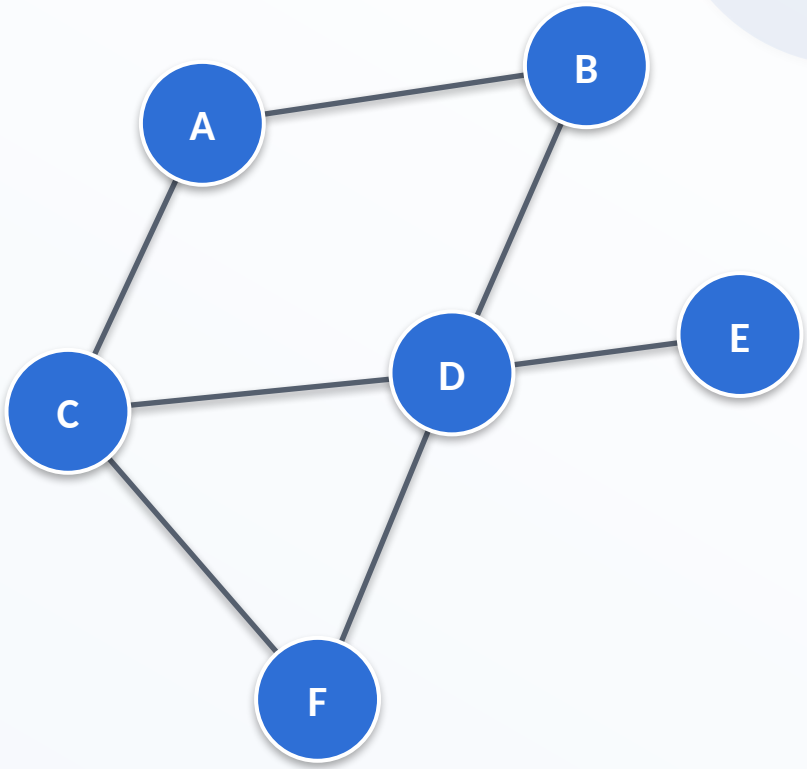
Depth First Search (DFS) • Breadth First Search (BFS)

A Simple Introduction



What is a Graph?

- A graph is a set of nodes (vertices) connected by edges
- It models relationships: cities–roads, people–friends, webpages–links
- Written as G = (V, E) → V = vertices, E = edges
- Example: Facebook friends, Google Maps routes



Types of Graphs

- Undirected Graph — edges have no direction (friendship)
- Directed Graph — edges have arrows (Twitter follows)
- Weighted Graph — edges have a cost/distance (road maps)
- Unweighted Graph — all edges are equal
- Cyclic / Acyclic — contains a cycle or not

How to Store a Graph?

- Adjacency Matrix — 2D table;
  matrix[i][j] = 1 if edge i→j exists
  - Good for dense graphs; edge check is O(1)
- Adjacency List — each node keeps a list of its neighbors
  - Good for sparse graphs; less memory
- Example (Adjacency List):

A → [B, C] B → [A, D]

C → [A, D] D → [B, C]

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

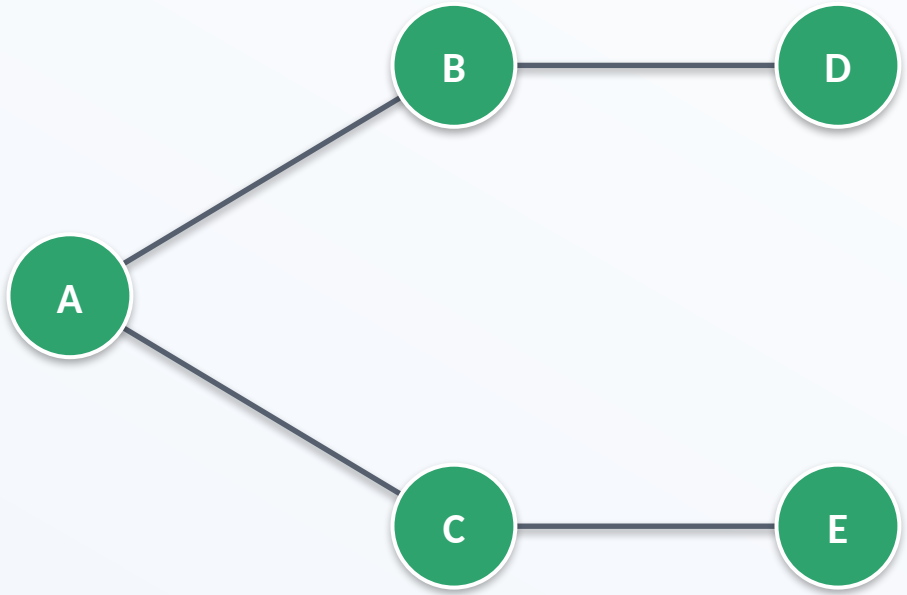
What is Graph Traversal?

- Traversal means visiting all nodes of a graph in a systematic way
- Two main techniques:
  - Depth First Search (DFS) — go deep first
  - Breadth First Search (BFS) — go level by level
- Used everywhere: path finding, network analysis, puzzles, compilers

Breadth First Search (BFS)

- Explores nodes level by level (like ripples in water)
- Uses a Queue (First In, First Out)
  - 1. Start at a node, mark it visited, push into queue
  - 2. Dequeue a node → visit it → enqueue unvisited neighbors
  - 3. Repeat until queue is empty
- Best for: shortest path in unweighted graphs, finding nearby nodes

BFS — Example (Start from A)



Order visited: A → B → C → D → E

Visit Order:

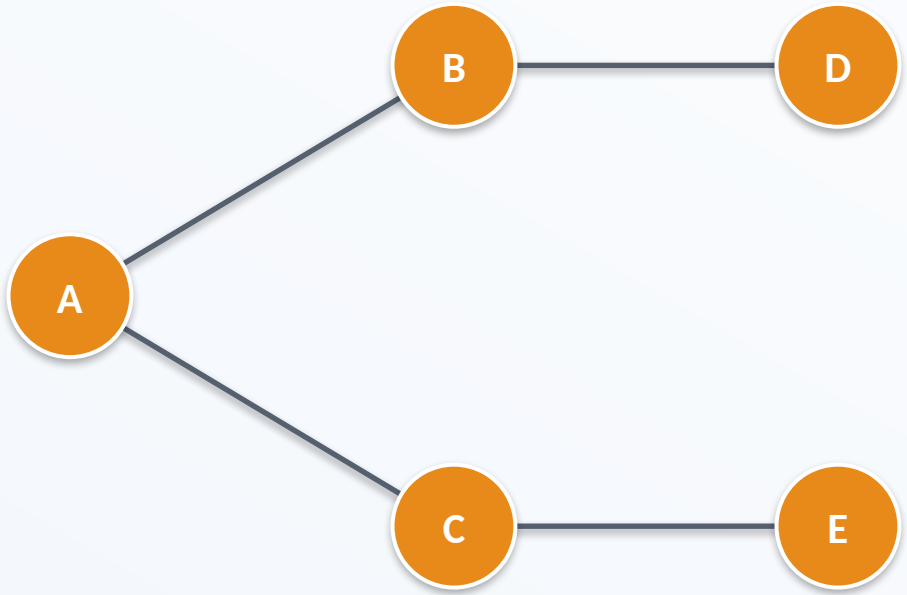
Level 0: A
Level 1: B, C
Level 2: D, E

Queue: A → B C → C D → D E → E → empty

Depth First Search (DFS)

- Explores as deep as possible, then backtracks
- Uses a Stack (or recursion — function call stack)
  - 1. Start at a node, mark it visited
  - 2. Go to an unvisited neighbor and repeat
  - 3. If stuck (no unvisited neighbors), backtrack
  - 4. Continue until all nodes are visited
- Best for: cycle detection, maze solving, topological sorting

DFS — Example (Start from A)



Order visited: A → B → D → C → E

How it goes:

A → B (deep!)
B → D
D has no new node,
backtrack to B,
backtrack to A
A → C → E

BFS vs DFS — Quick Comparison

Feature	BFS	DFS
Data structure	Queue	Stack / Recursion
Explores	Level by level	Depth first, then backtrack
Shortest path (unweighted)	Yes	No
Memory use	Can be high (many nodes per level)	Usually less
Best for	Nearby search, shortest path	Cycles, mazes, sorting tasks

Where Are They Used?

- Google Maps — shortest route (BFS idea)
- Social networks — friends of friends (BFS)
- Maze solving & puzzles like Sudoku (DFS)
- Web crawlers — index pages of the internet
- Compilers — dependency checking (DFS)
- AI & games — exploring possible moves

Summary

- Graph = nodes + edges, models real-world networks
- BFS explores level by level using a queue
- DFS goes deep first, then backtracks (stack/recursion)
- Both visit every node in O(V + E) time
- Choose BFS for shortest path, DFS for deep exploration

Thank You!

Created With

Kimi AI

by Moonshot AI (月之暗面)

AI-assisted presentation • Graphs, DFS & BFS

Thank You!